

Guía de Entrega y Documentación del Proyecto

Materia: Lógica de Programación — Aprendizaje Autónomo 2

Este documento contiene la estructura completa del proyecto de software desarrollado bajo los requerimientos de la actividad Autónoma 2. Incluye el planteamiento del problema, la lógica implementada, los diagramas de flujo y las pautas para generar los entregables en video y el repositorio de GitHub.

1. Configuración del Entorno de Desarrollo

Para el desarrollo de este sistema se ha seleccionado un entorno moderno y eficiente enfocado en la mantenibilidad del código:

- **Lenguaje de Programación:** Python 3.11+
- **IDE / Entorno de Desarrollo:** Visual Studio Code (VS Code)
- **Sistema de Control de Versiones:** Git y GitHub para el despliegue del repositorio.

2. Definición del Proyecto: Sistema de Gestión de Inventario y Descuentos

Se ha seleccionado un **Sistema de Gestión de Inventario y Caja Automatizada** para una tienda. Este problema del mundo real permite aplicar de forma profunda estructuras condicionales complejas y bucles repetitivos de control, cumpliendo estrictamente con la rúbrica de evaluación avanzada.

Funcionalidades Clave:

- **Control de Stock Operativo:** Permite ingresar múltiples artículos verificando si hay existencias disponibles.
 - **Cálculo Automático de Descuentos Dinámicos:** Aplica condicionales anidadas según el volumen de compra y tipo de cliente.
 - **Bucle de Control Total:** El sistema procesa productos de forma interactiva en un bucle continuo hasta que el usuario decida finalizar la transacción.
-

3. Implementación del Código Fuente (Python)

A continuación se presenta el código listo para ser copiado al archivo main.py dentro de tu entorno local:

```
#
=====
=====
# ACTIVIDAD: Aprendizaje Autónomo 2 – Lógica de Programación
# DESCRIPCIÓN: Sistema Automatizado de Gestión de Ventas e Inventario
# ARCHIVO: main.py
#
=====
=====

def ejecutar_sistema():
    # Inicialización de variables de stock simuladas
    stock_producto_A = 15
    stock_producto_B = 10

    precio_A = 25.00 # Precio unitario Producto A
    precio_B = 40.00 # Precio unitario Producto B

    total_venta = 0.0
    continuar_compra = "si"

    print("=== BIENVENIDO AL SISTEMA DE GESTIÓN DE VENTAS UIDE ===")

    # ESTRUCTURA REPETITIVA: Controla el flujo de la compra del
    cliente
    while continuar_compra.lower() == "si":
        print("\nProductos Disponibles:")
        print(f"1. Producto A - Precio: ${precio_A} (Stock:
{stock_producto_A})")
        print(f"2. Producto B - Precio: ${precio_B} (Stock:
{stock_producto_B})")

        opcion = input("\nSeleccione el número del producto que desea
comprar (1 o 2): ")

        # ESTRUCTURA CONDICIONAL: Validación y procesamiento de la
        selección
        if opcion == "1":
            cantidad = int(input(f"Ingrese la cantidad que desea del
Producto A (Máx {stock_producto_A}): "))

            # Condicional anidada para validar la disponibilidad del
            stock
```

```

        if cantidad <= stock_producto_A and cantidad > 0:
            subtotal_item = cantidad * precio_A
            stock_producto_A -= cantidad # Actualización
repetitiva del stock
            total_venta += subtotal_item
            print(f"-> Añadido con éxito. Subtotal:
${subtotal_item:.2f}")
        else:
            print("[ERROR] Cantidad no válida o supera el stock
disponible.")

    elif opcion == "2":
        cantidad = int(input(f"Ingrese la cantidad que desea del
Producto B (Máx {stock_producto_B}): "))

        if cantidad <= stock_producto_B and cantidad > 0:
            subtotal_item = cantidad * precio_B
            stock_producto_B -= cantidad
            total_venta += subtotal_item
            print(f"-> Añadido con éxito. Subtotal:
${subtotal_item:.2f}")
        else:
            print("[ERROR] Cantidad no válida o supera el stock
disponible.")

    else:
        print("[ALERTA] Opción de producto inválida. Intente
nuevamente.")

    # Pregunta de control para romper o continuar el bucle
destrutivo
    continuar_compra = input("\n¿Desea agregar otro producto al
carrito? (si/no): ")

    # ESTRUCTURA CONDICIONAL AVANZADA: Aplicación de descuentos por
volumen de facturación
    print("\n=====")
    print(f"Subtotal Bruto de la Venta: ${total_venta:.2f}")

    descuento = 0.0
    if total_venta >= 150.00:
        descuento = total_venta * 0.15 # 15% de descuento por compras
altas
        print(";Felicidades! Se aplicó un 15% de descuento por superar
los $150.")
    elif total_venta >= 50.00:
        descuento = total_venta * 0.05 # 5% de descuento
        print("Se aplicó un 5% de descuento por superar los $50.")

```

```

    else:
        print("No se registran descuentos aplicables para este
monto.")

    monto_final = total_venta - descuento
    print(f"Descuento Total: -${descuento:.2f}")
    print(f"TOTAL NETO A PAGAR: ${monto_final:.2f}")
    print("=====")
    print("Transacción finalizada con éxito. Inventario actualizado.")

if __name__ == "__main__":
    ejecutar_sistema()

```

4. Guía para los Videos Explicativos Obligatorios

La rúbrica exige la sustentación en formato audiovisual. Sigue este esquema para estructurar tus grabaciones de pantalla de manera fluida:

Video Parte 1: Configuración y Diseño

1. Muestra tu pantalla con la estructura de carpetas en tu computadora y el repositorio listo en GitHub.
2. Abre el diagrama de flujo y explica brevemente la lógica de control: cómo el flujo entra al bucle while y cómo se evalúa la disponibilidad del stock con los bloques condicionales if-else.

Video Parte 2: Demostración Funcional

1. Ejecuta el archivo main.py directo en la terminal de tu IDE.
2. Realiza dos pruebas dinámicas:
 - **Prueba 1:** Agrega un producto, selecciona "si" para añadir otro, genera un monto superior a \$150 para que el docente vea el cálculo dinámico del 15% de descuento y finaliza.
 - **Prueba 2:** Intenta comprar una cantidad que supere el stock actual (ej. 20 unidades) para demostrar que las alertas y el control de excepciones funcionan perfectamente sin romper el programa.